BLOCKCHAIN BASED RENTAL AGREEMENT

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Use Case Report

submitted by

Kannikanti Daiva prasanna aksa

23501A0561

Under the guidance of

Mr. A. Prashant, Asst. Prof.

Department of Computer Science and Engineering

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

2026-27

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

CERTIFICATE

This is to certify that the Use Case report entitled **"Blockchain Based Rental Agreement"** that is being submitted by **Kannikanti Daiva prasanna aksa (23501A0561)**, as part of Assignment-2 for the **Blockchain Technology (23CS4701B)** course in **4-1** during the academic year **2026-27**.

Course Coordinator
Mr. A. Prashant
Assistant Professor,
Department of CSE,
PVPSIT, Vijayawada

Head of the Department
Dr. A. Jayalakshmi,
Professor and Head,
Department of CSE,
PVPSIT, Vijayawada

MARKS

ASSIGNMENT-2: ____/5

INDEX

S. No.	Chapter	Page No.
1	Introduction	1
2	Background	2-3
3	Blockchain Basics	4-6
4	Use Case Overview	7-9
5	Implementation	10-16
6	Benefits	17-18
7	Challenges	19
8	Conclusion & Future Enhancements	20
9	SDG’s Addressed	21
10	References	22
11	Appendix A	23

1. INTRODUCTION

1.1. Overview of Blockchain Technology

Blockchain technology is a decentralized and distributed ledger system that records transactions across multiple nodes, ensuring transparency, security, and immutability. Unlike traditional databases that are controlled by a single organization, a blockchain operates on a peer-to-peer network in which every participating node holds a copy of the ledger. Each block contains a cryptographic hash of the previous block, so altering any historical record would require recomputing every subsequent block and obtaining the agreement of the network, which makes tampering practically infeasible [1].

Ethereum extended this idea beyond digital currency by introducing a general-purpose, programmable blockchain on which developers can deploy **smart contracts**: programs that live at an address on the chain and execute exactly as written when their conditions are met [2][3]. The concept of a smart contract, a computerised protocol that enforces the terms of an agreement without a trusted intermediary, was first articulated by Nick Szabo in the 1990s [5].

1.2. Relevance of Blockchain in Rental Agreement Systems

A rental agreement is one of the most common legal contracts in everyday life, yet it is still managed largely through printed stamp paper, handwritten receipts, bank statements, and verbal assurances. Landlords and tenants must depend on physical copies that can be lost, damaged, or altered, and on third parties such as brokers, notaries, and portal operators to establish trust.

A blockchain-based rental agreement system replaces the paper lease with a **digital agreement stored in a smart contract**. The landlord creates the agreement on-chain by specifying the property address, monthly rent, security deposit, and lease terms. The tenant reviews the stored terms and signs by calling the contract from their own wallet, producing a cryptographic signature that cannot be forged or later denied. Rent payments in India are largely made through off-chain channels such as UPI, net banking, and card gateways, which a smart contract cannot observe directly. To bridge this gap, the proposed system introduces a **Node.js Oracle service**. The oracle independently verifies a payment with the external payment gateway and, only when the transaction is confirmed, writes a payment record to the blockchain using the gateway reference as a unique key.

2. BACKGROUND

Traditional rental management relies on paper contracts, informal record keeping, and centralized rental portals. These arrangements create inefficiencies, invite fraud, and frequently lead to disagreements between landlords and tenants. Below are the key issues with existing rental systems:

2.1. Loss or Damage of Physical Rental Documents

Most residential leases are still executed on printed stamp paper and stored in cupboards or folders. Such documents can be misplaced, damaged by moisture or fire, or become illegible over time. When a dispute arises years later, either party may be unable to produce the original agreement, leaving them dependent on memory or on the other party's copy, which weakens their position.

2.2. Security Deposit Disputes and Unfair Withholding

The security deposit is the most frequent source of conflict in a tenancy. Landlords may withhold part or all of the deposit citing damages that the tenant disputes, while tenants may allege that the deposit was never returned in full. Since the deposit amount, deductions, and refund are often recorded only in informal messages or receipts, neither party can conclusively prove their claim.

2.3. Delayed Rent Tracking & Disputed Payment Receipts

Rent is paid through a mix of cash, bank transfers, and UPI, and the evidence is scattered across bank statements, screenshots, and handwritten slips. Landlords may claim that a month's rent was not received, whereas tenants may hold a screenshot that cannot be independently verified. Delays in reconciling payments make it difficult to track arrears accurately.

2.4. Lack of Transparency and Tampering of Lease Clauses

A paper lease or an editable digital file can be altered after signing, for example by replacing a page, changing the rent figure, or adding a clause. There is no built-in mechanism to prove which version was originally agreed upon, so tampering can be hard to detect and harder to prove.

2.5. Inefficiencies in Multi-party Brokerage and High Intermediary Fees

Brokers, agents, and rental platforms typically sit between landlord and tenant, charging fees that often equal one month's rent or more. Multi-party coordination between agents, owners, and tenants slows the process of drafting, verifying, and executing agreements, and adds cost without necessarily adding trust.

2.6. Fraudulent Subletting and Unauthorized Tenant Occupancy

Without a verifiable record of who is the legitimate tenant, properties can be sublet without the owner's knowledge, or fake leases can be produced to obtain deposits from unsuspecting occupants. Landlords struggle to prove who is authorised to occupy the premises, and genuine tenants may be victims of fraudulent listings.

2.7. Security Risks in Centralized Rental Portals and Data Breaches

Online rental portals store identity documents, contact details, and agreements in centralized databases. These repositories are attractive targets for cyberattacks, and a single breach can expose or corrupt the records of thousands of users. Users must also trust the portal operator not to modify or delete records.

2.8. Lengthy and Costly Legal Dispute Resolution

When a rental dispute cannot be settled informally, the parties must approach rent controllers or civil courts. Proceedings can last months or years and demand legal fees, repeated hearings, and the submission of documents whose authenticity may itself be contested.

2.9. Limited Access to Real-Time Rent Payment History

Tenants and landlords seldom have a single, up-to-date view of the payment history of a tenancy. Tenants who wish to demonstrate a clean payment record to future landlords or lenders have no portable, verifiable proof, while landlords must compile their own spreadsheets.

Due to these challenges, there is a growing need for a secure, transparent, and automated rental management system. Blockchain technology, combined with an oracle for off-chain payment verification, offers a promising solution by digitizing agreements, preserving immutable payment history, and removing reliance on intermediaries.

3. BLOCKCHAIN BASICS

Blockchain technology is a distributed ledger system that ensures secure, transparent, and tamper-proof transactions. Unlike traditional systems that rely on intermediaries, blockchain enables trustless peer-to-peer interactions, reducing the need for third parties in financial transactions, supply chain management, and property agreements. Below are the key concepts that define blockchain technology:

3.1. Decentralization

Traditional databases are managed by centralized authorities such as banks, portals, or corporations, creating a single point of failure that is vulnerable to hacks and data manipulation. A blockchain operates on a decentralized network in which multiple independent nodes validate and store data. No single entity has full control, which makes the system resistant to censorship and fraud. Compromising one node does not affect the integrity of the whole network, and since every transaction is verified across many nodes, participants can independently confirm it [1]. For a rental agreement, this means neither the landlord nor a portal operator can unilaterally edit or delete the lease.

3.2. Immutability

Immutability means that once a transaction is recorded it cannot be altered or deleted. This is achieved through cryptographic hashing: each block stores the hash of the preceding block, forming a chain in which tampering with one block invalidates all later blocks [1]. The integrity of the chain is protected by the consensus mechanism, which ensures that all honest participants agree before new data is appended. In the rental context, lease terms and payment records written to the contract remain permanently available as evidence.

3.3. Transparency and Auditability

Blockchain transactions and contract states are publicly verifiable by anyone with access to the network. Every participant holds an identical copy of the ledger, and every state change emits events that can be audited. For rental agreements, both parties can independently verify the agreed rent, the deposit, the current status, and every recorded payment without depending on the other party’s records [2].

3.4. Smart Contracts

Smart contracts are self-executing programs with predefined conditions that run on the blockchain [2][5]. Once the conditions are satisfied, the contract automatically performs the agreed action, eliminating manual processing and reducing disputes. In this project, the lease is modelled in Solidity as a **state machine** in which each agreement is in exactly one of the following states:

Table 3.1: Agreement Status Enumeration

State	Meaning
Pending	Agreement created by the landlord and awaiting the tenant's decision.
Accepted	Tenant has signed on-chain; the lease is active and rent payments can be recorded.
Rejected	Tenant declined the agreement; no further action is possible.
Expired	Tenant did not respond before the offer's expiry time.
Terminated	The active lease was ended by the landlord or the tenant.

Functions of the contract may be called only in the permitted states and by the permitted accounts, so illegal transitions (for example, recording rent against a rejected agreement) are rejected automatically by the Ethereum Virtual Machine [9].

3.5. Consensus Mechanisms

Blockchain networks use consensus mechanisms to validate transactions and keep the ledger consistent. Proof of Work (PoW), used by Bitcoin, requires miners to solve computational puzzles [1]. Proof of Stake (PoS), which Ethereum adopted after its transition to PoS in 2022, selects validators according to the ether they have staked, greatly reducing energy consumption [2][3].

Regardless of the consensus protocol, every node executes contract code deterministically in the **Ethereum Virtual Machine (EVM)**, so all honest nodes reach the same result. During development and testing, this project uses a **local Hardhat Network**, an in-process EVM-compatible blockchain that mines blocks instantly and offers pre-funded test accounts, so that the contract logic can be exercised without cost or delay [10]. The same contract can later be deployed to a public testnet or a Layer-2 network without changes to its logic.

3.6. Cryptographic Security & Asymmetric Digital Signatures

Blockchain security relies on public-key cryptography. Each user holds a private key, known only to them, and a public key from which their blockchain address is derived. Every transaction is signed with the private key using the Elliptic Curve Digital Signature Algorithm (ECDSA), and any node can verify the signature using the public key, proving that the transaction came from the owner of the address and was not altered in transit. Hash functions such as Keccak-256 convert data into fixed-length digests, which makes it infeasible to reverse-engineer the original data or to find two inputs with the same digest [3].

In this system, the tenant's call to the acceptance function is itself a digital signature on the agreement, binding their address to the exact terms stored on-chain. The oracle likewise signs the transaction that records each verified payment, which allows the contract to accept payment records only from the authorised oracle address.

By combining decentralization, immutability, transparency, smart contracts, and cryptographic security, blockchain provides a robust foundation for secure, automated, and verifiable rental agreements.

4. USE CASE OVERVIEW

The use case for a Blockchain Based Rental Agreement system aims to transform traditional lease management by leveraging blockchain technology and an oracle service. The system ensures secure, transparent, and fraud-resistant issuance of leases, tenant signing, rent payment verification, and lease termination.

4.1. Objectives

The primary objective is to **create digital, tamper-proof leases** that replace paper agreements. Once the landlord stores the property address, rent, security deposit, and terms in the smart contract, they cannot be modified without the network recording the change, so both parties always refer to the same authoritative version [7][8].

The second objective is **automated tenant signing**. The tenant reviews the terms and accepts or rejects the agreement directly from their wallet. The acceptance is a cryptographically signed transaction that cannot be repudiated, and the contract updates its status automatically.

The third objective is **external payment validation via an Oracle**. Because rent is paid through off-chain gateways, a Node.js oracle service checks each transaction with the gateway and only then records it on-chain. This links real-world fiat payments to the immutable ledger without requiring either party to trust the other's claim [12].

The fourth objective is **replay attack prevention**. Every gateway payment reference can be recorded only once. The contract maintains a mapping of processed references, so a previously used receipt cannot be submitted again to claim a second month's rent or to dispute an earlier payment.

4.2. Scope of the System

The system focuses on creating, accepting, tracking, and terminating residential rental agreements. The participating actors are:

1. **Landlord:** Creates rental agreements for owned properties, views the status of each agreement and its payment history, and may terminate an active lease.
2. **Tenant:** Reviews agreements addressed to their wallet, accepts or rejects them, pays rent through the payment gateway, and views their verified payment history.
3. **Node.js Oracle Service:** A trusted off-chain service that verifies gateway transactions and writes confirmed payments to the smart contract using a dedicated oracle account.

4. **Ethereum Blockchain Smart Contract:** The decentralized ledger and rule engine that stores agreements, enforces the status state machine, rejects duplicate payment references, and emits events for every action.

The scope excludes physical property inspection, legal registration of the lease with government authorities, and in-contract custody of funds; these are discussed as future enhancements in Chapter 8.

4.3. System Architecture

The architecture of the Blockchain Based Rental Agreement System is illustrated in Figure 4.1 and consists of four layers: the User Layer, the Web Application Layer, the Oracle Verification Layer, and the Blockchain Layer.

4.3.1. User Layer

This layer consists of the primary users who interact with the system (Figure 4.1 – User Layer).

- **Landlord:** creates agreements, views all issued agreements and their statuses, tracks verified rent payments, and terminates leases.
- **Tenant:** views agreements offered to them, accepts or rejects them, pays rent through the gateway, and checks their payment history.

4.3.2. Web Application Layer

This layer acts as the interface between users and the other layers (Figure 4.1 – Web Application Layer).

- **React.js Frontend:** provides dashboards for landlords and tenants, formats currency in INR, and uses ethers.js to read from and write to the smart contract through the user's wallet [11].
- **Node.js / Express API:** handles user registration and login with JSON Web Tokens (JWT), exposes REST endpoints for off-chain data, and coordinates with the oracle.
- **MongoDB:** stores user profiles, off-chain metadata such as display names and contact details, and a log of gateway transactions. Authoritative lease and payment data remain on-chain.

4.3.3. Oracle Verification Layer

This layer connects the blockchain to the off-chain financial world (Figure 4.1 – Oracle Verification Layer).

- **Node.js Oracle Service:** receives a payment notification or reference from the backend, queries the payment gateway to confirm that the transaction succeeded, validates the amount and agreement, and signs and submits a recordPayment transaction.
- **Payment Gateway:** the external source of truth for fiat transactions, such as UPI, net banking, or card payments.

4.3.4. Blockchain Layer

This layer ensures security, transparency, and immutability of agreements and payments (Figure 4.1 – Blockchain Layer).

- **RentalAgreement Smart Contract:** stores each agreement as a struct, enforces the status state machine, restricts sensitive functions to authorised accounts, and keeps a mapping of processed payment references.
- **Ethereum Network (Hardhat for development):** the distributed EVM-based ledger that executes the contract, maintains an immutable record of all transactions, and emits event logs that the frontend can subscribe to.

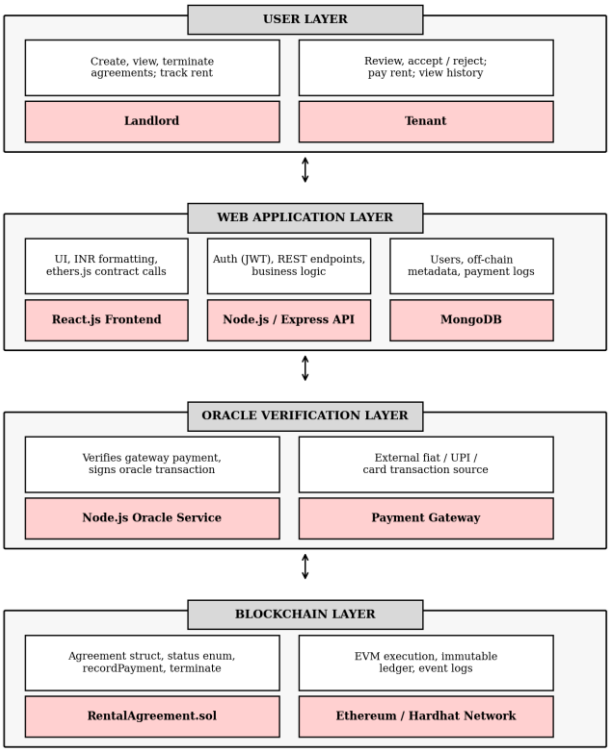


Figure 4.1: Architecture of Blockchain Based Rental Agreement System

5. IMPLEMENTATION

5.1. Setting Up the Blockchain Environment

To implement the system, the development environment is built around Ethereum technologies. **Hardhat** serves as the development framework for compiling, testing, and deploying smart contracts, and provides a local Hardhat Network with pre-funded accounts for rapid testing [10]. The smart contract is written in **Solidity ^0.8.24**, whose built-in checked arithmetic protects against integer overflow and underflow and whose custom errors and revert reasons support clear error handling [9]. The frontend is a **React.js** application that interacts with the contract using **ethers.js v6** [11], and a wallet such as MetaMask manages user accounts and signs transactions. The backend is an **Express.js** API running on Node.js, with MongoDB for off-chain data. The same Node.js runtime hosts the oracle service, which uses ethers.js to sign transactions with a dedicated oracle account.

Table 5.1: Technology Stack

Component	Technology / Purpose
Smart contract language	Solidity ^0.8.24
Development framework	Hardhat (compile, test, deploy, local network)
Blockchain library	ethers.js v6 (Contract, Provider, Signer)
Frontend	React.js with INR currency formatting
Backend API	Node.js with Express, JWT authentication
Database	MongoDB (off-chain user data and logs)
Oracle	Node.js service with a dedicated oracle wallet
Wallet	MetaMask for transaction signing

5.2. Defining Smart Contracts

5.2.1. Agreement Struct & State Variables

The RentalAgreement contract defines an **Agreement** struct that holds all essential details: the agreement ID, property address, landlord and tenant addresses, rent amount, security deposit, lease terms, and a status field. The status field uses an enumeration with the five states Pending, Accepted, Rejected, Expired, and Terminated. State variables include the oracle address, a counter for the next agreement ID, a mapping from ID to agreement, and the **isPaymentProcessed** mapping that tracks which payment references have already been

recorded. Rent and deposit values are stored as integers in the smallest unit (paise) to avoid floating-point errors, and are displayed in rupees by the frontend.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract RentalAgreement {
    enum Status { Pending, Accepted, Rejected, Expired, Terminated }

    struct Agreement {
        uint256 agreementId;
        string  propertyAddress;
        address landlord;
        address tenant;
        uint256 rentAmount;       // in paise
        uint256 securityDeposit;  // in paise
        string  terms;
        uint256 createdAt;
        uint256 expiresAt;        // offer validity
        Status  status;
    }

    struct Payment {
        bytes32 paymentRef;
        uint256 amount;
        uint256 timestamp;
    }

    address public oracle;
    uint256 public nextAgreementId = 1;
    mapping(uint256 => Agreement) public agreements;
    mapping(uint256 => Payment[]) private payments;
    mapping(bytes32 => bool) public isPaymentProcessed;

    event AgreementCreated(uint256 indexed id,
                           address indexed landlord, address indexed tenant);
    event AgreementAccepted(uint256 indexed id);
    event AgreementRejected(uint256 indexed id);
    event PaymentRecorded(uint256 indexed id, bytes32 ref, uint256 amount);
    event AgreementTerminated(uint256 indexed id, address by);

    constructor(address _oracle) { oracle = _oracle; }
    modifier onlyOracle() { require(msg.sender == oracle, "Only oracle"); _; }
}
```

Code 5.1: Agreement struct and state variables

5.2.2. Agreement Creation by Landlord

The landlord creates a new agreement by calling **createAgreement**, supplying the tenant's address, property address, rent, security deposit, terms, and the validity period of the offer. The function checks that the tenant address is valid and different from the landlord, assigns a unique agreement ID, stores the struct with status Pending, and emits an AgreementCreated event that the frontend uses to notify the tenant.

```
function createAgreement(
    address tenant,
```

```
    string memory propertyAddress,
    uint256 rentAmount,
    uint256 securityDeposit,
    string memory terms,
    uint256 validForSeconds
) external returns (uint256) {
    require(tenant != address(0) && tenant != msg.sender, "Invalid tenant");
    require(rentAmount > 0, "Rent must be > 0");

    uint256 id = nextAgreementId++;
    agreements[id] = Agreement({
        agreementId: id,
        propertyAddress: propertyAddress,
        landlord: msg.sender,
        tenant: tenant,
        rentAmount: rentAmount,
        securityDeposit: securityDeposit,
        terms: terms,
        createdAt: block.timestamp,
        expiresAt: block.timestamp + validForSeconds,
        status: Status.Pending
    });
    emit AgreementCreated(id, msg.sender, tenant);
    return id;
}
```

Code 5.2: createAgreement function

5.2.3. Agreement Acceptance & Rejection by Tenant

Only the tenant named in the agreement may accept or reject it, and only while the agreement is Pending and the offer has not expired. Acceptance changes the status to Accepted and constitutes the tenant's digital signature on the stored terms. Rejection sets the status to Rejected. If the tenant does not respond before expiry, any account can mark the offer as Expired through a helper function, after which accept and reject calls fail.

```
function acceptAgreement(uint256 id) external {
    Agreement storage a = agreements[id];
    require(msg.sender == a.tenant, "Not the tenant");
    require(a.status == Status.Pending, "Not pending");
    require(block.timestamp <= a.expiresAt, "Offer expired");
    a.status = Status.Accepted;
    emit AgreementAccepted(id);
}

function rejectAgreement(uint256 id) external {
    Agreement storage a = agreements[id];
    require(msg.sender == a.tenant, "Not the tenant");
    require(a.status == Status.Pending, "Not pending");
    a.status = Status.Rejected;
    emit AgreementRejected(id);
}

function expireAgreement(uint256 id) external {
    Agreement storage a = agreements[id];
    require(a.status == Status.Pending, "Not pending");
    require(block.timestamp > a.expiresAt, "Not yet expired");
    a.status = Status.Expired;
}
```

Code 5.3: acceptAgreement, rejectAgreement and expireAgreement functions

5.2.4. Node.js Oracle Verification Service & External Payment Gateway Handling

Smart contracts cannot call external web services, so a Node.js oracle service is used to bring verified off-chain payment information on-chain [12]. When a tenant pays rent through the payment gateway, the backend receives the gateway's transaction reference. The oracle then performs the following steps:

1. Queries the payment gateway's API with the transaction reference and confirms that the payment status is successful.
2. Validates the paid amount against the rent amount stored in the agreement, and confirms that the agreement is in the Accepted state.
3. Checks the isPaymentProcessed mapping (via a read call) to ensure that the reference has not already been recorded.
4. Hashes the gateway reference using Keccak-256 to obtain a fixed-size bytes32 key.
5. Signs and submits a recordPayment transaction using the dedicated oracle wallet, whose address was supplied to the contract at deployment.

```
// oracle/verifyAndRecord.js (simplified)
const { ethers } = require('ethers');

async function verifyAndRecord(agreementId, gatewayRef) {
  // 1. Ask the payment gateway for the transaction details
  const txn = await gateway.fetchPayment(gatewayRef);
  if (!txn || txn.status !== 'SUCCESS') throw new Error('Payment not verified');

  // 2. Validate against the on-chain agreement
  const a = await contract.agreements(agreementId);
  if (Number(a.status) !== 1) throw new Error('Agreement not active');
  if (txn.amountPaise !== a.rentAmount) throw new Error('Amount mismatch');

  // 3. Replay protection check before spending gas
  const ref = ethers.id(gatewayRef);               // keccak256 hash
  if (await contract.isPaymentProcessed(ref)) throw new Error('Duplicate
reference');

  // 4. Sign and submit with the oracle wallet
  const tx = await contractAsOracle.recordPayment(agreementId, ref,
txn.amountPaise);
  await tx.wait();
  return tx.hash;
}
```

Code 5.4: Oracle verification and submission (Node.js, ethers.js v6)

Because only the oracle address can call recordPayment, neither landlord nor tenant can fabricate a payment. Because the oracle verifies the gateway response before writing, the on-chain record reflects real, confirmed transactions.

5.2.5. Recording Rent Payments On-Chain

The **recordPayment** function is restricted by the onlyOracle modifier. It requires that the agreement is Accepted, that the payment reference has not been processed before, and that the amount matches the agreed rent. It then marks the reference as processed in the **isPaymentProcessed** mapping, appends the payment to the agreement's history, and emits a PaymentRecorded event. This duplicate-reference check provides **replay attack prevention**: even if the same valid receipt is submitted repeatedly, only the first submission succeeds.

```
function recordPayment(
    uint256 id,
    bytes32 paymentRef,
    uint256 amount
) external onlyOracle {
    Agreement storage a = agreements[id];
    require(a.status == Status.Accepted, "Agreement not active");
    require(!isPaymentProcessed[paymentRef], "Payment already processed");
    require(amount == a.rentAmount, "Incorrect amount");

    isPaymentProcessed[paymentRef] = true;
    payments[id].push(Payment(paymentRef, amount, block.timestamp));
    emit PaymentRecorded(id, paymentRef, amount);
}

function getPayments(uint256 id) external view returns (Payment[] memory) {
    return payments[id];
}
```

Code 5.5: recordPayment function with duplicate reference prevention

5.2.6. Agreement Termination

An active lease may be ended by either the landlord or the tenant. The **terminateAgreement** function verifies that the caller is a party to the agreement and that the status is Accepted, then sets the status to Terminated and emits an AgreementTerminated event. The complete lease history, including all recorded payments, remains permanently readable after termination, which provides an auditable record for deposit settlement.

```
function terminateAgreement(uint256 id) external {
    Agreement storage a = agreements[id];
    require(
        msg.sender == a.landlord || msg.sender == a.tenant,
        "Not a party to agreement"
    );
    require(a.status == Status.Accepted, "Agreement not active");
    a.status = Status.Terminated;
    emit AgreementTerminated(id, msg.sender);
}
```

Code 5.6: terminateAgreement function

5.3. Deploying and Integrating with Frontend

After the contract is compiled and tested with Hardhat, it is deployed to the local Hardhat Network using a deployment script that passes the oracle account's address to the constructor. The deployment output provides the contract address and ABI, which are placed in the frontend configuration. The same process can later be repeated against a public test network such as Sepolia before any production deployment.

The frontend is built with **React.js** and uses **ethers.js v6** to create a BrowserProvider from the user's wallet, obtain a Signer, and instantiate a Contract object from the address and ABI. Read-only calls such as fetching an agreement or payment list use the provider, whereas state-changing calls such as createAgreement and acceptAgreement are signed by the user's wallet. Because the contract stores rent and deposit in paise, the interface converts these to rupees and formats them using the Indian locale (for example, ₹15,000.00) through the browser's Intl.NumberFormat API. Dashboards are role-specific: landlords see the agreements they created and a payment ledger, while tenants see pending offers, active leases, and their verified payment history.

```
import { BrowserProvider, Contract } from 'ethers';

const provider = new BrowserProvider(window.ethereum);
const signer   = await provider.getSigner();
const rental   = new Contract(CONTRACT_ADDRESS, ABI, signer);

const inr = new Intl.NumberFormat('en-IN',
  { style: 'currency', currency: 'INR' });

// Tenant accepts an agreement
const tx = await rental.acceptAgreement(agreementId);
await tx.wait();

// Display rent in rupees
const a = await rental.agreements(agreementId);
console.log(inr.format(Number(a.rentAmount) / 100));
```

Code 5.7: Frontend integration with ethers.js

5.4. Workflow & Sequence

The workflow of the Blockchain Based Rental Agreement System, as illustrated in Figure 5.1, details the interactions between the landlord, tenant, oracle, payment gateway, and smart contract.

The process begins with the landlord creating an agreement for a property and a named tenant. The contract stores the agreement with status Pending and emits an event. The tenant reviews the terms on-chain and either accepts, rejects, or allows the offer to expire. A rejection or expiry

ends the flow, while acceptance moves the agreement to the Accepted state and activates the lease.

During the lease, the tenant pays each month's rent through the external payment gateway in INR. The oracle then fetches the transaction from the gateway and verifies its status, amount, and reference. If the transaction is valid and its reference has not been used before, the oracle calls recordPayment, which marks the reference as processed and emits a PaymentRecorded event so that both parties' dashboards update. If the transaction is invalid or its reference is a duplicate, the oracle discards it and nothing is written to the blockchain. At the end of the tenancy, or if either party wishes to end it earlier, terminateAgreement sets the status to Terminated.

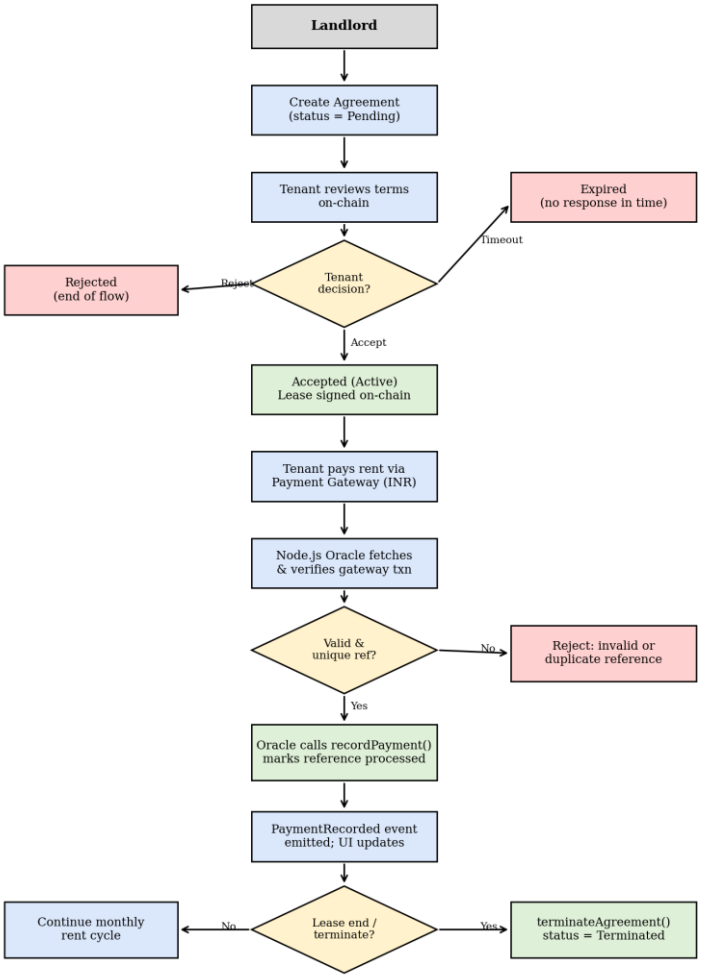


Figure 5.1: Workflow of Blockchain Based Rental Agreement with Oracle Payment Verification

6. BENEFITS

Blockchain technology enhances rental management by introducing transparency, security, and automation. Traditional systems depend on paper records and centralized databases, which are prone to fraud, inefficiency, and data loss. By combining a smart contract with oracle-verified payments, rental management becomes tamper-proof, cost-effective, and efficient. The key advantages are:

6.1. Enhanced Security and Immutability

Once an agreement or payment is recorded on the blockchain, it cannot be altered or deleted. Lease terms remain exactly as agreed, and the cryptographic protection of the ledger reduces the risk of hacking and unauthorized modification [1][3].

6.2. Elimination of Fraudulent and Duplicate Rent Claims

Each gateway payment reference can be recorded only once, so the same receipt cannot be reused to claim rent twice or to show payment for a month that was not paid. Because only the oracle can record payments, neither landlord nor tenant can fabricate entries.

6.3. Security Deposit Transparency and Protection

The security deposit amount is stored in the agreement and visible to both parties for the whole tenancy. This removes ambiguity about what was agreed and provides an objective basis for deposit settlement at the end of the lease, reducing the scope for unfair withholding.

6.4. Autonomous Oracle-Verified Payment Confirmation

Payments are confirmed automatically by the oracle through direct verification with the payment gateway, without manual reconciliation. Landlords receive timely, trustworthy confirmation, and tenants receive an immutable proof of payment.

6.5. Elimination of Intermediary Brokers and Administrative Overhead

Because the agreement, signing, and payment tracking are handled by the smart contract and oracle, there is less need for brokers, notaries, and manual record keepers. This lowers fees and speeds up onboarding [8].

6.6. Cryptographically Verifiable Audit Ledger

Every action produces a signed transaction and an event log with a timestamp. The resulting ledger can be independently verified by any party, including courts or auditors, without trusting either the landlord or the platform operator.

6.7. Role-Based Access Control and Tenant Privacy

Smart contract functions are restricted by role: only landlords create agreements, only the named tenant can accept or reject, and only the oracle can record payments. Personal details such as identity proofs and contact information are kept off-chain in the database, so that the public ledger holds only the data required to enforce the lease.

6.8. Reduction of Tenancy Disputes and Legal Costs

Because the terms, status, and payment history are objective and shared, many disputes can be resolved by simply consulting the ledger. Where litigation is unavoidable, the immutable records serve as strong, easily verifiable evidence, shortening proceedings and reducing costs.

6.9. Streamlined Lease Lifecycle from Issuance to Termination

The complete lifecycle of a tenancy, namely creation, signing, rent collection, and termination, is managed in one system with clear state transitions. Both parties always know the current status of the lease and what actions are permitted.

6.10. Sustainability through 100% Paperless Real Estate Management

Replacing printed leases, receipts, and notices with digital records reduces paper consumption and physical storage, supporting environmentally responsible real estate management.

Implementing blockchain in rental management brings security, efficiency, and transparency to the process. It prevents fraud, lowers administrative costs, and improves the experience of both landlords and tenants by making agreements verifiable, payments auditable, and the lease lifecycle automated.

7. CHALLENGES

While blockchain-based rental systems offer many advantages, there are challenges and limitations that must be considered for effective implementation. Key challenges include:

7.1. Scalability and Gas Fees on Public Blockchains

On public blockchains, every state-changing transaction costs gas, and fees fluctuate with network congestion. Recording monthly rent payments for many agreements could become expensive on Ethereum mainnet. Layer-2 networks, batching of payment records, or permissioned deployments can mitigate this cost [6].

7.2. Oracle Reliability & Security of Off-Chain Payment Gateways

The oracle is a trusted component: if its private key is compromised, or if the payment gateway returns incorrect data, false payments could be recorded. The oracle therefore needs secure key management, monitoring, and rate limiting. Decentralized oracle networks or multi-signature verification can reduce this single point of trust [12].

7.3. Legal and Regulatory Compliance for Digital Real Estate Leases

Rental law, stamp duty, and registration requirements differ across states and countries. In India, leases beyond a certain duration require registration, and the legal recognition of smart contracts is still evolving. Data-protection obligations also restrict what personal information may be placed on an immutable ledger.

7.4. User Awareness and Adoption of Web3 Wallets

Many landlords and tenants are unfamiliar with wallets, private keys, and transaction signing. Losing a private key can mean losing access to an agreement. User education, simplified onboarding, and wallet recovery options are required for wider adoption.

7.5. Internet Connectivity and Infrastructure Constraints

Interacting with the blockchain and the payment gateway requires reliable internet access. Users in areas with poor connectivity may find it difficult to sign agreements or verify payments in time.

Despite these challenges, blockchain-based rental management offers transformative potential. Addressing them requires regulatory alignment, careful oracle design, security audits, user education, and the adoption of scalable blockchain solutions.

8. CONCLUSION

This report explored the integration of blockchain technology into rental agreement management, addressing the key problems of traditional systems: loss of paper documents, security deposit disputes, disputed payment receipts, tampering of lease clauses, brokerage costs, and costly dispute resolution.

The implemented system uses a Solidity smart contract, deployed on a Hardhat network, to represent each lease as a state machine with the states Pending, Accepted, Rejected, Expired, and Terminated. Landlords create agreements, tenants sign them with their wallets, and a Node.js oracle verifies off-chain gateway payments before recording them on-chain. A mapping of processed payment references prevents replay attacks and duplicate claims. Overall, the system achieves digital tamper-proof leases, automated signing, trustworthy payment history, and reduced dependency on intermediaries.

8.1. Future Outlook for Enhancements

Several enhancements could further improve the system. **Decentralized Identity (DID)** can be integrated for tenant KYC, allowing tenants to prove their identity and creditworthiness using verifiable credentials without storing personal documents on a central server. This would also help prevent fraudulent subletting.

IoT smart lock integration can link physical access to on-chain rent status: when rent is recorded as paid, the smart lock continues to grant access, and when a lease is terminated the access rights are revoked automatically.

Multi-signature escrow contracts can hold security deposits so that funds are released only with the agreement of both parties, or through a neutral arbitrator in case of dispute. This would move deposit handling fully on-chain and strengthen protection against unfair withholding.

Layer-2 rollup integration, such as Optimistic or zero-knowledge rollups, can substantially reduce gas costs and increase throughput, making monthly payment recording practical at scale. Additional work could include decentralized oracle networks, multi-chain interoperability, and automated rent reminders and renewal notifications.

By continuously improving the system with these advancements, blockchain-based rental management can evolve into a more secure, efficient, and widely adopted solution for the real estate sector.

9. SDG's ADDRESSED

The Blockchain Based Rental Agreement system aligns with several United Nations Sustainable Development Goals (SDGs) by enhancing transparency, reducing paper use, improving institutional trust, and promoting fair access to housing [13]. The key SDGs addressed are:

SDG 9: Industry, Innovation, and Infrastructure

The system promotes digital transformation of the real estate sector by applying blockchain, smart contracts, and oracle technology to a traditionally paper-based process. This builds resilient digital infrastructure and encourages technological innovation in property management.

SDG 11: Sustainable Cities and Communities

Fair, transparent, and accessible rental housing is essential for sustainable cities. By protecting tenants from unfair deposit withholding and landlords from non-payment, and by reducing the cost of renting, the system contributes to safe and affordable housing arrangements.

SDG 16: Peace, Justice, and Strong Institutions

By eliminating fraud, providing transparent and unalterable covenants, and reducing tenancy disputes, the system supports access to justice and builds trust in institutions. Objective, verifiable evidence reduces the burden on courts and helps resolve conflicts more peacefully.

By integrating blockchain, the Blockchain Based Rental Agreement system fosters a secure, transparent, and sustainable ecosystem for managing rental housing.

10. REFERENCES

[1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, “A Next-Generation Smart Contract and Decentralized Application Platform (Ethereum White Paper),” 2014. [Online]. Available: https://ethereum.org/en/whitepaper/

[3] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper),” Ethereum Project, 2014. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[4] K. Christidis and M. Devetsikiotis, “Blockchains and Smart Contracts for the Internet of Things,” IEEE Access, vol. 4, pp. 2292–2303, 2016.

[5] N. Szabo, “Formalizing and Securing Relationships on Public Networks,” First Monday, vol. 2, no. 9, Sep. 1997.

[6] Z. Zheng, S. Xie, H.-N. Dai, X. Chen, and H. Wang, “Blockchain challenges and opportunities: a survey,” International Journal of Web and Grid Services, vol. 14, no. 4, pp. 352–375, 2018.

11. APPENDIX A

Project Repository

GitHub repository: https://github.com/aksakannikanti/blockchain-rental-agreement